UNIVERSITÉ HASSAN II DE CASABLANCA

FACULTÉ DES SCIENCES BEN M'SICK

DÉPARTEMENT DE MATHÉMATIQUES ET INFORMATIQUE

Master d'excellence en Intelligence Artificielle

RAPPORT DE PROJET DE FIN DE MODULE :

Mise en place d'une infrastructure DevOps avec CI/CD, Kubernetes, et Monitoring

Réalisé par :

Mr.Hassan el hadi

Mlle.BERRIMA MERIEM

Mlle.AMEJOUD HASNAE

Mlle.BENTAIB Hajar

Exposé le 24 février 2026 devant les membres de jury :

Pr. F. benabbou	Faculté des sciences Ben M'Sick	Encadrante

Année Universitaire : 2025 - 2026

Dédicace

À nos très chers parents,

Aucune dédicace ne saurait exprimer l'ampleur de l'amour et de la gratitude que nous vous portons. Vous avez consacré votre vie à notre éducation, nous avez appris la persévérance et le sens du devoir, et vous n'avez jamais cessé de croire en nous. Ce travail est avant tout le fruit de vos sacrifices et de votre amour inconditionnel.

À nos amis et collègues de la promotion Master d’excellence en intelligence artificielle,

Pour les heures de travail partagées, les échanges intellectuels et les moments de solidarité qui ont rendu cette aventure humainement inoubliable. Vous avez été une véritable source de motivation dans les moments de doute comme dans ceux de réussite. Que cette amitié forgée dans les salles de cours et les nuits de travail perdurent bien au-delà de cette promotion.

À nos enseignants, tout au long de notre parcours,

Votre passion pour la transmission du savoir, votre rigueur et votre dévouement ont façonné notre façon de penser et d'aborder les problèmes. Chacun d'entre vous a contribué, à sa manière, à faire de nous les étudiants que nous sommes aujourd'hui. Nous espérons, à travers ce travail, vous rendre ne serait-ce qu'une infime partie de ce que vous nous avez donné.

Remerciements

Avant tout, nous remercions **Dieu** de nous avoir accordé la santé, la volonté et le courage nécessaires pour mener à bien ce travail. C'est grâce à Sa grâce que nous avons pu surmonter les difficultés rencontrées tout au long de ce projet et aller jusqu'au bout de nos ambitions.

Nous adressons également nos sincères remerciements à notre **professeur f.benabbou**, pour la qualité remarquable de l'enseignement dispensé dans le cadre du module Python pour la Data Science. Grâce à sa pédagogie claire, sa passion pour la matière et sa capacité à rendre accessibles des concepts complexes, nous avons pu acquérir les compétences techniques et méthodologiques qui ont constitué le socle de ce projet. .

Nos remerciements s'étendent à l'ensemble du **corps professoral du Département Informatique de la Faculté des Sciences Ben M'Sick (FSBM)** de l'Université Hassan II de Casablanca, pour la richesse de la formation qu'ils dispensent avec dévouement et professionnalisme. Chaque cours, chaque atelier et chaque projet encadré au sein de cette institution a contribué à forger notre profil de futurs ingénieurs et data scientists. Nous sommes fiers d'appartenir à cet établissement et reconnaissants envers ceux qui œuvrent quotidiennement à maintenir son niveau d'excellence.

Enfin, nous tenons à nous féliciter mutuellement en tant que membres de cette équipe **, Hasan el hadi, Hasnae Amjoud, Hajar Bentaib et Meriem Berrima** pour l'esprit de collaboration, la complémentarité et l'engagement sincère dont chacun a fait preuve tout au long de ce projet. Les moments de travail intensif, les débats techniques et les efforts partagés ont non seulement enrichi ce travail, mais ont également renforcé les liens qui nous unissent en tant que groupe. Ce projet est avant tout le fruit d'un travail d'équipe dont nous sommes collectivement fiers.

À toutes ces personnes, nous exprimons notre plus profonde et sincère reconnaissance.

Sommaire

Dédicace 2
Remerciements 3
Sommaire 4
Liste des figures 6
Listes des abréviations 8
1.1 Contexte 10
1.2 Problématique 10
1.3 Objectifs du projet 10
1.4 Démarche adoptée 11
2.1 Sujet du projet 13
2.2 Exigences du cahier des charges 13
2.3 Architecture globale 13
2.4 Schéma fonctionnel 13
3.1 Introduction 15
3.2 Choix du provider cloud 15
3.3 Configuration du provider 15
3.4 Création du Security Group 16
3.5 Création des instances EC2 16
Instance Worker 17
3.6 Analyse des paramètres 18
3.7 Variables et outputs 18
3.8 Commandes Terraform 19
..... 19
3.9 Résultats obtenus 19
3.10 Limites et améliorations 20
3.11 Conclusion 20
Introduction 22
4.2 Préparation de l'environnement 22
4.3 Installation de Docker 23
4.4 Installation de Kubernetes 24
4.5 Initialisation du cluster 24
4.6 Ajout du worker node 25
4.7 Configuration du réseau 25

4.8 Test du cluster 26
4.9 SCHÉMA D'ARCHITECTURE 26
4.10 conclusion 26
5.1 Introduction 28
5.2 Description de l'application 28
5.3 Code source de l'application 28
5.4 Fichier des dépendances 28
5.5 Dockerfile 29
5.6 Construction de l'image 29
5.7 Exécution locale du conteneur 30
5.8 Tests de l'application 31
5.9 Conclusion 32
6.1 Introduction 34
6.2 Manifests utilisés 34
Deployment 34
6.3 Déploiement sur le cluster 35
6.4 Image Docker utilisée 36
6.5 Problème rencontré 36
6.6 Validation du service 36
6.7 Tests applicatifs sur Minikube 36
6.8 Conclusion 38
7.1 Introduction 40
7.2 Outil choisi 40
7.3 Étapes du pipeline 40
7.4 Structure du dépôt 40
7.5 Apport du pipeline 41
7.6 Conclusion 41
8.1 Validation de l'application locale 43
8.2 Validation du conteneur Docker 43
8.3 Validation Kubernetes 43
8.4 Validation du pipeline CI/CD 43
8.5 Interprétation des résultats 43
Conclusion générale et perspectives 44
References 45

Liste des figures

Figure 1 : Sélection de l'Amazon Machine Image (AMI) pour le déploiement des instances EC2
Figure 2 : Instances EC2 Kubernetes Master et Worker en cours d'exécution
Figure 3 : Résultat de la commande Terraform Plan montrant les ressources à créer
Figure 4 : Exécution de Terraform Apply et récupération des adresses IP publiques

Figure 5 : Exemple du fichier inventory Ansible
Figure 6 : Organisation des fichiers Ansible du projet
Figure 7 : Vérification de la connectivité Ansible avec les nœuds Kubernetes

Figure 8 : Installation de Docker sur les serveurs
Figure 9 : Installation des composants Kubernetes

Figure 10 : Adresse IP du serveur Master utilisée pour l'ajout du Worker

Figure 11 : Construction de l'image Docker avec succès
Figure 12 : Image Docker créée avec succès (devops-flask-api)
Figure 13 : Lancement du conteneur Docker exécutant l'application Flask
Figure 14 : Conteneur Docker en cours d'exécution (docker ps)
Figure 15 : Test de l'endpoint principal (/) de l'API
Figure 16 : Test de l'endpoint /health retournant le statut de l'application

Figure 17 : Publication de l'image Docker de l'application Flask sur Docker Hub
Figure 18 : Exposition du service Kubernetes via NodePort
Figure 19 : Manifest Kubernetes ConfigMap pour la configuration de l'application

Figure 20 : Image Docker publique disponible sur Docker Hub
Figure 21 : Exposition du service Kubernetes avec la commande Minikube
Figure 22 : Vérification du fonctionnement de l'application déployée
Figure 23 : Accès au service hello-minikube exposé sur le cluster
Figure 24 : Validation du déploiement Kubernetes et informations du Pod

Figure 25 : Exécution du pipeline CI/CD avec GitHub Action

Listes des abréviations

AMI : Amazon Machine Image
API : Application Programming Interface
AWS : Amazon Web Services
CI : Continuous Integration
CI/CD : Continuous Integration / Continuous Deployment
CPU : Central Processing Unit
DNS : Domain Name System
EC2 : Elastic Compute Cloud
Flask : Framework léger Python pour le développement d'applications web
Git : Global Information Tracker
HCL : HashiCorp Configuration Language
HTTP : HyperText Transfer Protocol
IaC : Infrastructure as Code
IP : Internet Protocol
JSON : JavaScript Object Notation
K8s : Kubernetes
kubeadm : Outil d'initialisation d'un cluster Kubernetes
kubectl : Outil en ligne de commande pour gérer Kubernetes
NodePort : Type de service Kubernetes exposant une application sur un port du nœud
Pod : Plus petite unité déployable dans Kubernetes
REST : Representational State Transfer
SSH : Secure Shell
URL : Uniform Resource Locator
VM : Virtual Machine
YAML : YAML Ain't Markup Language

● Chapitre 1 : Introduction Générale et Contexte du Projet

1.1 Contexte

Avec l'évolution rapide des systèmes d'information, les entreprises cherchent à accélérer le développement et le déploiement des applications tout en garantissant la stabilité, la sécurité et la disponibilité des services. Dans ce contexte, l'approche DevOps joue un rôle fondamental en rapprochant les équipes de développement et d'exploitation autour d'objectifs communs : automatisation, intégration continue, déploiement continu et supervision.

Le DevOps ne se limite pas à un simple ensemble d'outils. Il s'agit d'une méthodologie qui favorise la collaboration, la reproductibilité des environnements, l'automatisation des tâches répétitives et l'amélioration continue du cycle de vie logiciel.

1.2 Problématique

Le déploiement manuel d'une application sur plusieurs serveurs est souvent long, source d'erreurs et difficile à reproduire. Dans un environnement moderne, il devient nécessaire d'automatiser :

la création de l'infrastructure

la configuration des machines

la conteneurisation de l'application

l'orchestration des services

la livraison continue du code

Le projet proposé répond précisément à cette problématique en demandant la mise en place d'une infrastructure DevOps complète sur une plateforme cloud, avec un minimum de deux machines virtuelles, une application conteneurisée, un cluster Kubernetes et une chaîne CI/CD.

1.3 Objectifs du projet

Le projet a pour objectif principal de construire une infrastructure complète permettant de déployer automatiquement une application conteneurisée sur le cloud. Plus précisément, il s'agit de :

créer une infrastructure cloud sur AWS

provisionner au moins deux machines virtuelles pour former un cluster Kubernetes

- développer une application simple
- conteneuriser cette application avec Docker
- déployer l'application dans Kubernetes
- mettre en place une chaîne CI/CD pour automatiser le build et la publication
- documenter l'ensemble du processus de manière claire et reproductible

Ces objectifs sont cohérents avec le cahier des charges officiel, qui impose Terraform, Ansible, Kubernetes, une application conteneurisée et un pipeline CI/CD.

1.4 Démarche adoptée

Pour répondre à ces objectifs, le projet a été structuré en plusieurs étapes :

- provisionnement de l'infrastructure cloud avec Terraform
- préparation de la configuration des nœuds
- développement et conteneurisation de l'application avec Docker
- déploiement sur Kubernetes à l'aide de manifests YAML
- automatisation du build et du push avec GitHub Actions
- validation du déploiement à travers différents tests

● Chapitre 2 : Présentation generale du projet

2.1 Sujet du projet

Le sujet consiste à mettre en place une infrastructure DevOps complète sur une machine cloud, en utilisant les technologies modernes d'automatisation, de conteneurisation, d'orchestration et de déploiement continu. L'objectif est de simuler une infrastructure de bout en bout permettant de déployer une application conteneurisée à travers une chaîne CI/CD automatisée.

2.2 Exigences du cahier des charges

Le cahier des charges impose notamment :

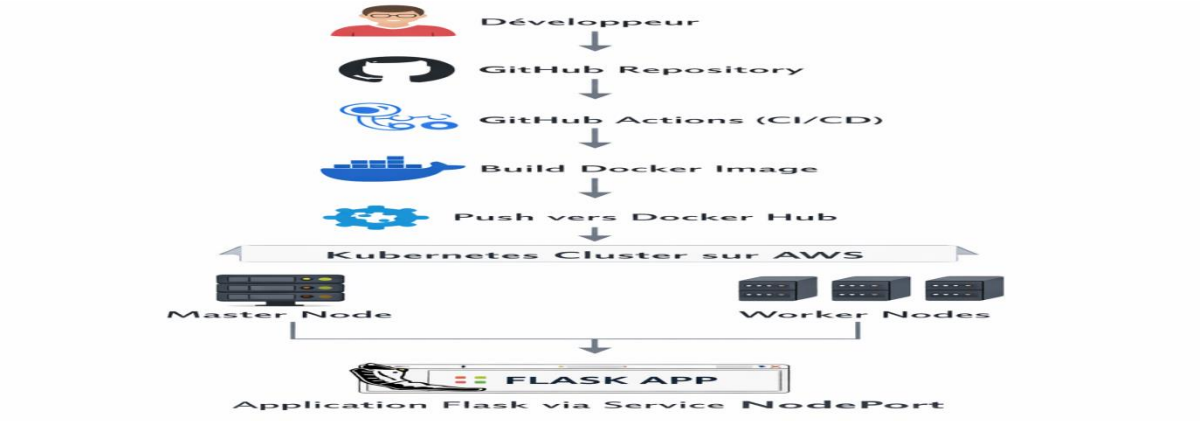
- le déploiement d'une application simple
- la conteneurisation de l'application avec Docker
- l'utilisation d'un cluster Kubernetes
- la création d'au moins deux machines virtuelles cloud
- l'automatisation avec Terraform et Ansible
- la mise en place d'une chaîne CI/CD
- la documentation complète du processus

2.3 Architecture globale

L'architecture globale du projet repose sur les composants suivants :

- **AWS** comme provider cloud
- **Terraform** pour le provisioning des ressources
- **Ansible** pour la configuration automatisée
- **Docker** pour la conteneurisation
- **Kubernetes** pour l'orchestration
- **Docker Hub** comme registre d'images
- **GitHub Actions** pour le pipeline CI/CD
- **Flask** pour l'application de démonstration

2.4 Schéma fonctionnel



● Chapitre 3 : Mise en place de l’infrastructure cloud avec Terraform

3.1 Introduction

La première étape technique du projet a consisté à provisionner l'infrastructure cloud nécessaire à l'hébergement du cluster Kubernetes. Conformément aux exigences du sujet, l'infrastructure devait comprendre au minimum deux machines virtuelles, un nœud maître et un nœud worker.

Pour répondre à ce besoin, l'outil **Terraform** a été utilisé. Terraform permet d'adopter une approche **Infrastructure as Code**, dans laquelle les ressources cloud sont décrites sous forme de code déclaratif.

3.2 Choix du provider cloud

Le provider choisi est **AWS**, avec la région :

eu-north-1

Ce choix permet de bénéficier d'un environnement cloud largement utilisé dans le domaine DevOps et compatible avec les services nécessaires au projet.

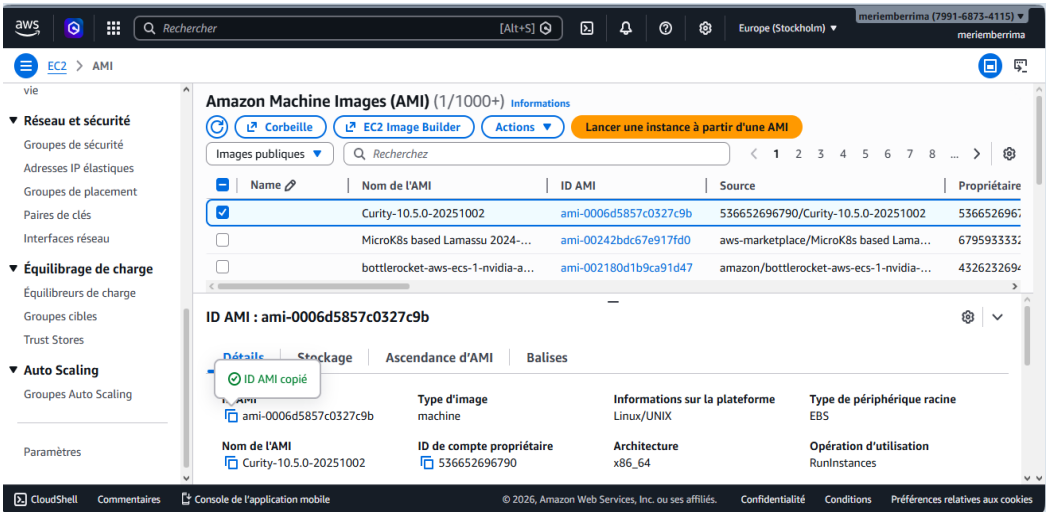


Figure 1 Sélection de l'Amazon Machine Image (AMI) pour le déploiement des instances EC2

3.3 Configuration du provider

La configuration du provider AWS est la suivante :

```
provider "aws" {
region = "eu-north-1"
}
```

Cette déclaration indique à Terraform de créer l'ensemble des ressources dans la région **eu-north-1**.

3.4 Création du Security Group

Pour autoriser l'accès distant aux machines, un groupe de sécurité a été défini :

```
resource "aws_security_group"
"ssh_access" {
name        = "allow_ssh"
description = "Allow SSH inbound
traffic"

ingress {
from_port   = 22
to_port     = 22
protocol    = "tcp"
cidr_blocks = ["0.0.0.0/0"]
}

egress {
from_port   = 0
to_port     = 0
protocol    = "-1"
cidr_blocks = ["0.0.0.0/0"]
}
}
```

Ce groupe de sécurité permet :

- l'accès SSH entrant sur le port 22
- tout le trafic sortant

Il s'agit d'une configuration pratique dans un cadre académique, même si, dans un contexte de production, il serait préférable de restreindre l'accès SSH à une adresse IP de confiance.

3.5 Création des instances EC2

Deux instances EC2 ont été créées :

- **kubernetes-master**
- **kubernetes-worker**

Instance Master

```
resource "aws_instance" "master" {
ami = "ami-0028b2c25f5ca28ed"
instance_type = "t3.micro"
key_name = "devops-key2"
vpc_security_group_ids = [aws_security_group.ssh_access.id]

tags = {
Name = "kubernetes-master"
}
}
```

Instance Worker

```
resource "aws_instance" "worker" {
ami = "ami-0028b2c25f5ca28ed"
instance_type = "t3.micro"
key_name = "devops-key2"
vpc_security_group_ids = [aws_security_group.ssh_access.id]

tags = {
Name = "kubernetes-worker"
}
}
```

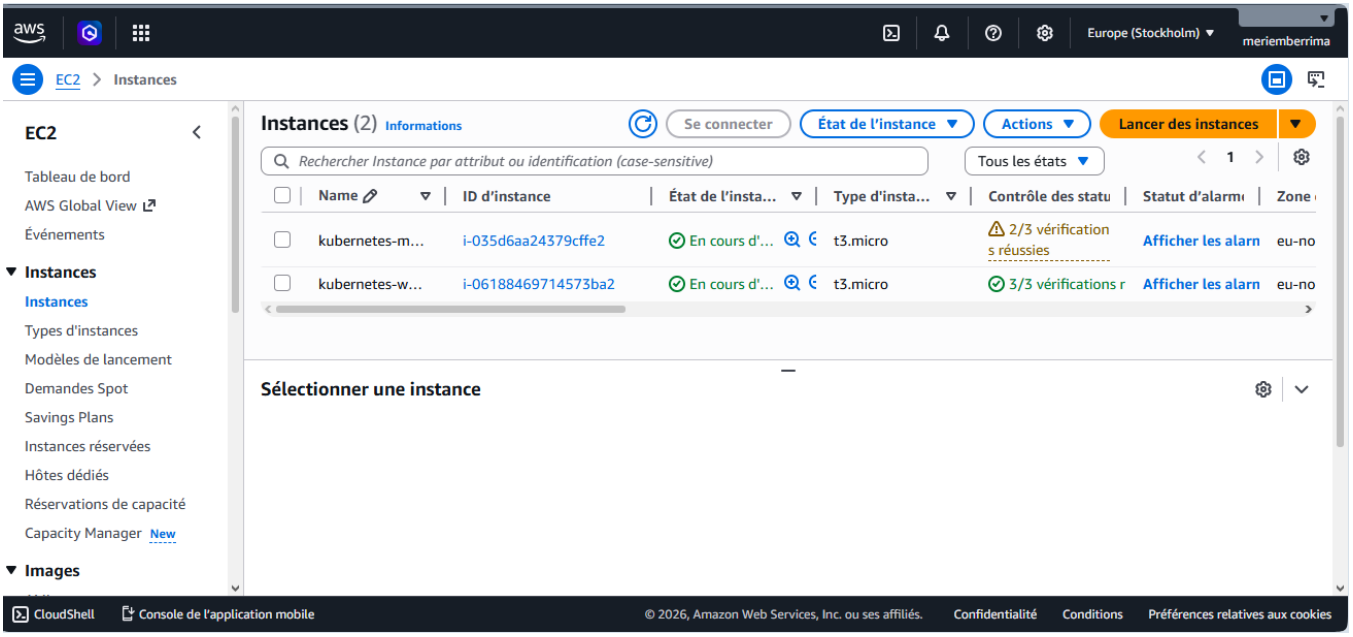


Figure 2Instances EC2 Kubernetes Master et Worker en cours d'exécution

3.6 Analyse des paramètres

Les paramètres principaux sont les suivants :

AMI : ami-0028b2c25f5ca28ed
instance_type : t3.micro
key_name : devops-key2
security group : allow_ssh

3.7 Variables et outputs

- Des variables ont été définies :

```
variable "region" {
default = "us-east-1"
}

variable "instance_type" {
default = "t2.micro"
}
```

- Des outputs ont aussi été configurés pour récupérer les IP publiques :
- output "master_ip" {

```
value = aws_instance.master.public_ip
}
```

```
output "worker_ip" {
value = aws_instance.worker.public_ip
}
```

- Ces outputs facilitent l'accès aux machines après exécution de Terraform.

3.8 Commandes Terraform

```
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\WINDOWS\system32>C:\Users\pc\Desktop\terraform-k8s-project\terraform
'C:\Users\pc\Desktop\terraform-k8s-project\terraform' n'est pas reconnu en tant que commande interne
ou externe, un programme exécutable ou un fichier de commandes.

C:\WINDOWS\system32>cd C:\Users\pc\Desktop\terraform-k8s-project\terraform

C:\Users\pc\Desktop\terraform-k8s-project\terraform>terraform apply

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.master will be created
  + resource "aws_instance" "master" {
      + ami                                  = "ami-0006d5857c0327c9b"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "devops-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
```

Figure 3Résultat de la commande Terraform Plan montrant les ressources à créer

Les principales commandes utilisées sont :

```
terraform init
terraform plan
terraform apply
```

- `terraform init` initialise le projet et télécharge le provider AWS
- `terraform plan` affiche le plan d'exécution
- `terraform apply` crée les ressources effectivement

3.9 Résultats obtenus

```
            # (1 unchanged attribute hidden)
        },
      ]
    + name                   = "allow_ssh"
    + name_prefix            = (known after apply)
    + owner_id               = (known after apply)
    + region                 = "eu-north-1"
    + revoke_rules_on_delete = false
    + tags_all               = (known after apply)
    + vpc_id                 = (known after apply)
  }

Plan: 3 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + master_ip = (known after apply)
  + worker_ip = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.ssh_access: Creating...
aws_security_group.ssh_access: Creation complete after 7s [id=sg-07f3f9c1c695c15d8]
aws_instance.master: Creating...
aws_instance.worker: Creating...
aws_instance.worker: Still creating... [00m10s elapsed]
aws_instance.master: Still creating... [00m10s elapsed]
aws_instance.worker: Creation complete after 15s [id=i-06188469714573ba2]
aws_instance.master: Creation complete after 17s [id=i-035d6aa24379cffe2]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

master_ip = "13.60.245.49"
worker_ip = "13.63.161.45"

C:\Users\pc\Desktop\terraform-k8s-project\terraform>
```

Figure 4Exécution de Terraform Apply et récupération des adresses IP publiques

Après exécution du script Terraform, l'infrastructure obtenue comprend :

- un groupe de sécurité SSH
- une instance EC2 Master
- une instance EC2 Worker
- les IP publiques des deux machines

Cette infrastructure constitue la base du cluster Kubernetes demandé dans le projet.

3.10 Limites et améliorations

- Quelques améliorations possibles :
- utiliser les variables réellement dans les ressources pour éviter le hardcoding
- ouvrir les ports nécessaires à Kubernetes et à l'application
- restreindre le SSH à une IP donnée
- modulariser le code Terraform

3.11 Conclusion

Terraform a permis d'automatiser la création de l'infrastructure cloud de manière claire, reproductible et rapide. Cette étape constitue la base de tout le projet, car elle prépare l'environnement nécessaire au déploiement de Kubernetes et de l'application.

● Chapitre 4 : Configuration et automatisation avec Ansible

Introduction

Dans cette partie, l'objectif principal est d'automatiser la configuration des serveurs afin de préparer un environnement prêt à accueillir un cluster Kubernetes.

Pour cela, l'outil Ansible a été utilisé. Il s'agit d'un outil d'automatisation qui permet d'exécuter des tâches de configuration sur plusieurs machines à distance sans intervention manuelle. Grâce à son fonctionnement basé sur SSH, aucune installation d'agent n'est nécessaire sur les machines cibles, ce qui simplifie considérablement la gestion.

L'utilisation d'Ansible dans ce projet permet de :

- réduire les erreurs humaines
- accélérer le déploiement
- garantir une configuration identique sur toutes les machines

Les serveurs ont été configurés pour installer les outils essentiels tels que Docker pour la conteneurisation et Kubernetes pour l'orchestration.

4.2 Préparation de l'environnement

Avant d'exécuter les playbooks, une phase de préparation a été nécessaire.

Tout d'abord, Ansible a été installé sur la machine locale en utilisant un environnement Linux via WSL. Cela permet de travailler dans un environnement compatible avec les outils DevOps.

Ensuite, une connexion sécurisée a été mise en place entre la machine locale et les serveurs distants via SSH, en utilisant une clé privée. Cette méthode garantit un accès sécurisé sans mot de passe.

Un fichier d'inventaire a été créé pour définir les machines cibles. Ce fichier joue un rôle central dans Ansible, car il permet d'organiser les serveurs en groupes (master et worker) et de faciliter l'exécution des commandes.

Cette étape est essentielle car elle permet à Ansible de savoir :

- sur quelles machines exécuter les tâches
- avec quel utilisateur se con

```
hajar@Hajar-Bentaib:~/devops-project$ ls ansible/
install_docker.yml      inventory          terraform-key.pem
install_kubernetes.yml  setup_master.yml
erraform-key.pem

[worker]
13.63.161.45 ansible_user=bitnami ansible_ssh_private_key_file=t
erraform-key.pem
```

```
hajar@Hajar-Bentaib:~/devops-project$ ls ansible/
install_docker.yml      inventory          terraform-key.pem
install_kubernetes.yml  setup_master.yml
```

Figure 5 exemple cat inventory

Figure 6 les fichier ansible

```
hajar@Hajar-Bentaib:~/devops-project/ansible$ ansible all -i inv
entory -m ping
13.63.161.45 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
13.60.245.49 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

Figure 7Vérification de la connectivité Ansible avec les nœuds Kubernetes master et worker

4.3 Installation de Docker

L'installation de Docker constitue une étape fondamentale, car Kubernetes repose sur les conteneurs pour exécuter les applications.

Un playbook Ansible a été créé pour automatiser cette installation sur tous les serveurs. Ce playbook effectue plusieurs actions importantes :

- mise à jour des paquets système
- installation du moteur Docker
- démarrage du service Docker
- activation du service au démarrage du système

Grâce à cette automatisation, Docker est installé de manière uniforme sur toutes les machines, ce qui garantit la compatibilité avec Kubernetes.

```
hajar@Hajar-Bentaib:~/devops-project/ansible$ ansible-playbook -
i inventory install_docker.yml

PLAY [Install Docker on all servers] ******************************
*****************

TASK [Gathering Facts] ********************************************
*****************
ok: [13.63.161.45]
ok: [13.60.245.49]

TASK [Update apt packages] ****************************************
*****************
ok: [13.63.161.45]
changed: [13.60.245.49]

TASK [Install Docker] *********************************************
*****************
ok: [13.63.161.45]
ok: [13.60.245.49]

TASK [Start Docker service] ***************************************
*****************
ok: [13.63.161.45]
ok: [13.60.245.49]

PLAY RECAP ********************************************************
*****************
13.60.245.49               : ok=4    changed=1    unreachable=0
    failed=0    skipped=0    rescued=0    ignored=0
13.63.161.45               : ok=4    changed=0    unreachable=0
```

Figure 8 installation docker

4.4 Installation de Kubernetes

Après Docker, l'étape suivante consiste à installer les composants de Kubernetes.

Trois outils principaux sont installés :

- kubeadm : pour initialiser et gérer le cluster
- kubelet : agent qui s'exécute sur chaque nœud
- kubectl : outil de gestion du cluster

Le playbook Ansible permet également de :

- ajouter les dépôts officiels Kubernetes
- installer les dépendances nécessaires
- configurer les services système

```
hajar@Hajar-Bentaib:~/devops-project/ansible$ ansible-playbook -
i inventory install_kubernetes.yml

PLAY [Installation simple de Kubernetes sur Ubuntu 24.04] ******
****************

TASK [Gathering Facts] ********************************************
****************
ok: [13.63.161.45]
ok: [13.60.245.49]

TASK [Désactiver swap] ********************************************
****************
changed: [13.63.161.45]
changed: [13.60.245.49]

TASK [Installer containerd] ***************************************
****************
ok: [13.63.161.45]
```

Figure 9 installation cubernetes

Cette étape prépare les serveurs à devenir des nœuds Kubernetes.

4.5 Initialisation du cluster

Le cluster Kubernetes est initialisé sur le serveur maître à l'aide de l'outil kubeadm.

Cette étape est cruciale, car elle permet de :

- créer le plan de contrôle (API Server, Scheduler, Controller Manager)
- générer les certificats de sécurité
- configurer la communication entre les composants

Une fois l'initialisation terminée, un fichier de configuration est généré pour permettre l'utilisation de kubectl.

4.6 Ajout du worker node

Après la création du cluster, un nœud worker est ajouté afin de distribuer la charge de travail.

Le worker se connecte au cluster en exécutant une commande générée par kubeadm. Cette commande contient :

- l'adresse du master

```
hajar@Hajar-Bentaib:~/devops-project/ansible$ ansible-playbook -
i inventory setup_master.yml
[WARNING]: While constructing a mapping from /home/hajar/devops-
project/ansible/setup_master.yml, line 6, column 7, found a dupl
icate dict key
(register). Using last defined value only.
[WARNING]: While constructing a mapping from /home/hajar/devops-
project/ansible/setup_master.yml, line 6, column 7, found a dupl
icate dict key
(args). Using last defined value only.

PLAY [Initialisation du master Kubernetes] *********************
****************

TASK [Gathering Facts] *****************************************
****************
```

Figure 10 addresse master

- un token de sécurité
- un hash pour la vérification

Une fois connecté, le worker devient capable d'exécuter des pods.

4.7 Configuration du réseau

Pour permettre la communication entre les conteneurs, un plugin réseau est nécessaire.

Dans ce projet, Flannel a été utilisé. Il crée un réseau virtuel permettant aux pods de communiquer entre eux, même s'ils sont sur des machines différentes.

Sans cette étape, les pods ne pourraient pas interagir entre eux.

4.8 Test du cluster

Afin de valider le bon fonctionnement du cluster, une application de test a été déployée.

Un déploiement nginx a été créé, puis exposé via un service de type NodePort. Cela permet d'accéder à l'application depuis l'extérieur du cluster.

Les tests ont confirmé que :

- les pods sont en état *Running*
- le worker exécute correctement les conteneurs
- le cluster fonctionne normalement

4.9 SCHÉMA D'ARCHITECTURE

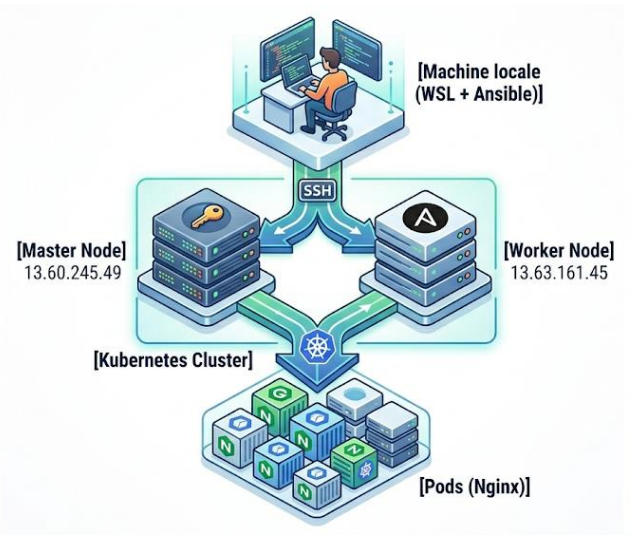


Figure 11 : architecture

4.10 conclusion

Dans cette partie, nous avons mis en place une configuration automatisée des serveurs à l'aide de l'outil Ansible. Cette automatisation a permis de simplifier le processus d'installation et de garantir une configuration cohérente sur l'ensemble des machines.

Les outils Docker et Kubernetes ont été installés avec succès, et un cluster fonctionnel a été déployé comprenant un nœud maître et un nœud worker.

Les tests réalisés ont permis de valider le bon fonctionnement du cluster, notamment à travers le déploiement d'une application nginx. Cette étape constitue une base solide pour la suite du projet, qui consiste à déployer une application réelle et à automatiser le processus via un pipeline CI/CD.

● Chapitre 5 : Développement et conteneurisation de l’application

5.1 Introduction

Dans cette partie, une application simple a été développée sous forme d'API REST avec Flask, puis conteneurisée à l'aide de Docker afin de faciliter son exécution et son déploiement sur Kubernetes.

5.2 Description de l'application

L'application développée est une API REST simple basée sur Flask. Elle expose deux endpoints principaux :

- `/` : retourne un message de bienvenue
- `/health` : retourne l'état de l'application

Cette API a été choisie comme cas d'étude simple permettant de démontrer l'intégration dans une architecture DevOps.

5.3 Code source de l'application

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return {"message": "Bienvenue sur
l'API DevOps"}

@app.route("/health")
def health():
    return {"status": "OK"}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Ce code permet d'exposer un service HTTP très léger, suffisant pour valider le déploiement.

5.4 Fichier des dépendances

Le fichier `requirements.txt` contient :

```
flask
```

Il permet d'installer les dépendances Python nécessaires à l'exécution de l'application.

5.5 Dockerfile

Le Dockerfile utilisé est le suivant :

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r
requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Ce Dockerfile définit un environnement minimal basé sur Python 3.12, copie les fichiers de l'application, installe les dépendances, expose le port 5000 et lance l'application Flask.

5.6 Construction de l'image

La construction de l'image Docker se fait avec la commande :

docker build -t devops-flask-api .

Cette étape prépare une image exécutable contenant l'application et toutes ses dépendances.

```
PS C:\Users\PC\OneDrive\Desktop\devops-project> docker build -t devops-flask-api .
>>
[+] Building 4.4s (10/10) FINISHED                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                        0.2s
 => => transferring dockerfile: 217B                                        0.1s
 => [internal] load metadata for docker.io/library/python:3.12-slim         2.1s
 => [internal] load .dockerignore                                           0.1s
 => => transferring context: 2B                                             0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:3d5ed973e45820f5ba5  0.2s
 => => resolve docker.io/library/python:3.12-slim@sha256:3d5ed973e45820f5ba5  0.1s
 => [internal] load build context                                           0.1s
 => => transferring context: 63B                                            0.0s
 => CACHED [2/5] WORKDIR /app                                               0.0s
 => CACHED [3/5] COPY requirements.txt .                                    0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt         0.0s
 => CACHED [5/5] COPY app.py .                                              0.0s
 => exporting to image                                                      0.7s
 => => exporting layers                                                     0.0s
 => => exporting manifest sha256:5e469640af9cb6a11eec39ae67e8261b8261c50a61c  0.0s
 => => exporting config sha256:5ee82bf60041e1901eeb20c321d885c234ae39ea920f1  0.0s
 => => exporting attestation manifest sha256:1cb078ead19c41435a4d463714eb8c0  0.2s
 => => exporting manifest list sha256:1e5baedf0881c566cdd4604e90dbcdc905eb67  0.1s
 => => naming to docker.io/library/devops-flask-api:latest                  0.0s
 => => unpacking to docker.io/library/devops-flask-api:latest               0.1s
```

Figure 11Construction de l'image Docker avec succès

```
PS C:\Users\PC\OneDrive\Desktop\devops-project> docker images
                                                  i Info →   U   In Use
IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
devops-flask-api:latest
                         1e5baedf0881       197MB         48.1MB      U
```

Figure 12Image Docker créée avec succès (devops-flask-api)

5.7 Exécution locale du conteneur

Le conteneur est lancé avec la commande :

```
docker run -p 5000:5000 devops-flask-api
```

L'application devient alors accessible localement via :

http://localhost:5000

```
PS C:\Users\PC\OneDrive\Desktop\devops-project> docker run -p 5000:5000 devops-flas
k-api
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Us
e a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
```

Figure 13 Lancement du conteneur Docker exécutant l'application Flask

```
PS C:\Users\PC\OneDrive\Desktop\devops-project> docker ps
CONTAINER ID   IMAGE              COMMAND           CREATED          STATUS          PORTS
                     NAMES
2e52ea1e6a88   devops-flask-api   "python app.py"   15 minutes ago   Up 15 minutes   0.0.0.0:5000->5000/tcp,
[::]:5000->5000/tcp   pedantic_heisenberg
```

Figure 14 Conteneur Docker en cours d'exécution (docker ps)

5.8 Tests de l'application

Les endpoints ont été testés avec succès :

http://localhost:5000

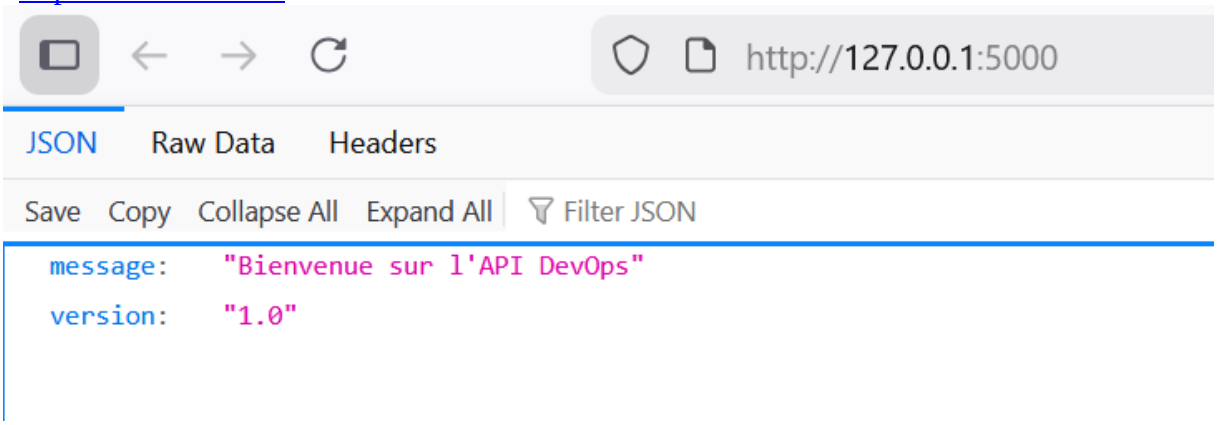


Figure 15 Test de l'endpoint principal (/) de l'API

http://localhost:5000/health

Les réponses JSON confirment le bon fonctionnement de l'API et la validité de la conteneurisation.

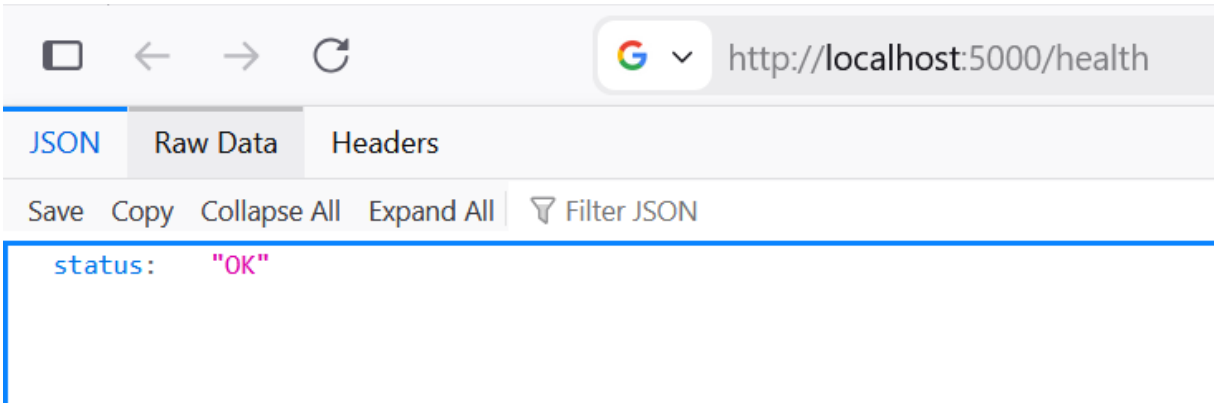


Figure 16 Test de l'endpoint /health retournant le statut de l'application

5.9 Conclusion

Cette étape a permis de disposer d'une application simple, conteneurisée et prête à être intégrée dans la chaîne DevOps du projet. Elle constitue la charge applicative qui sera déployée sur Kubernetes.

● Chapitre 6: Orchestration et déploiement avec Kubernetes

6.1 Introduction

Kubernetes est utilisé dans ce projet pour orchestrer l'exécution de l'application conteneurisée. Son rôle est d'assurer la disponibilité, la reproductibilité et l'exposition du service.

Le projet comprend des manifests Kubernetes de type Deployment, Service et ConfigMap.

6.2 Manifests utilisés

Deployment

Le `Deployment` gère le cycle de vie des pods. Il définit :

- l'image Docker à utiliser
- le nombre de réplicas
- les ports exposés
- les ressources allouées au conteneur

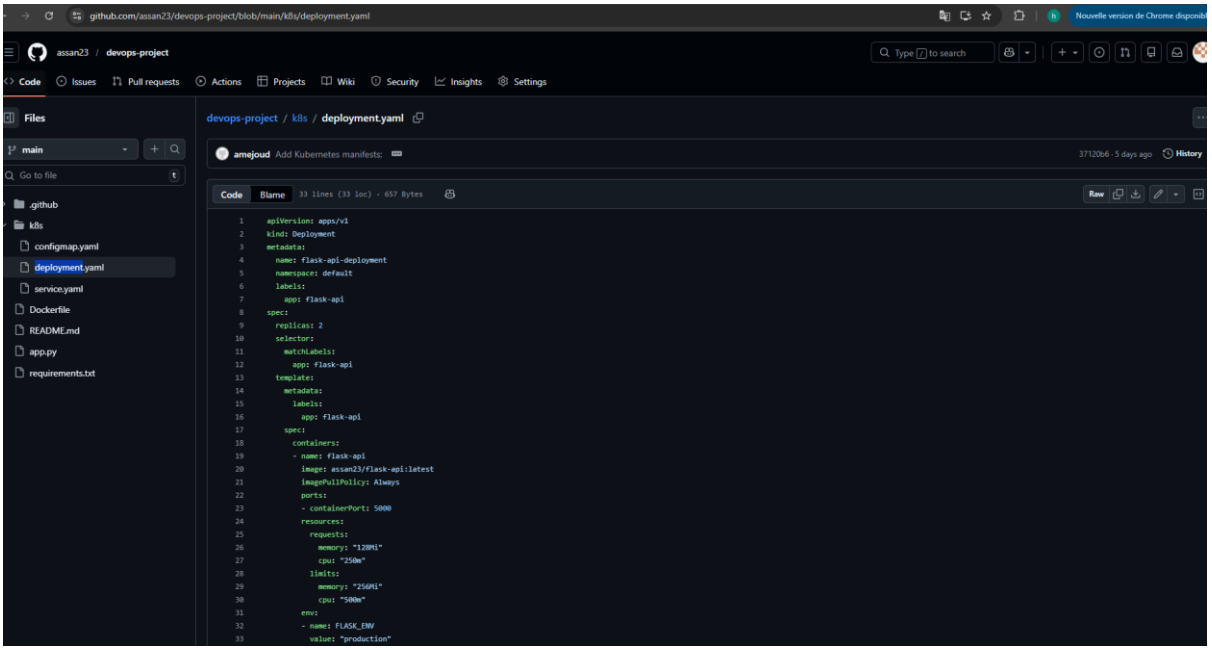


Figure 17Dépôt de l'image Docker de l'application Flask sur Docker Hub

Son rôle est de maintenir le nombre souhaité de pods et de faciliter les mises à jour.

Service

Le Service expose l'application, ici via un type NodePort, afin de permettre un accès externe via l'IP publique d'un nœud.

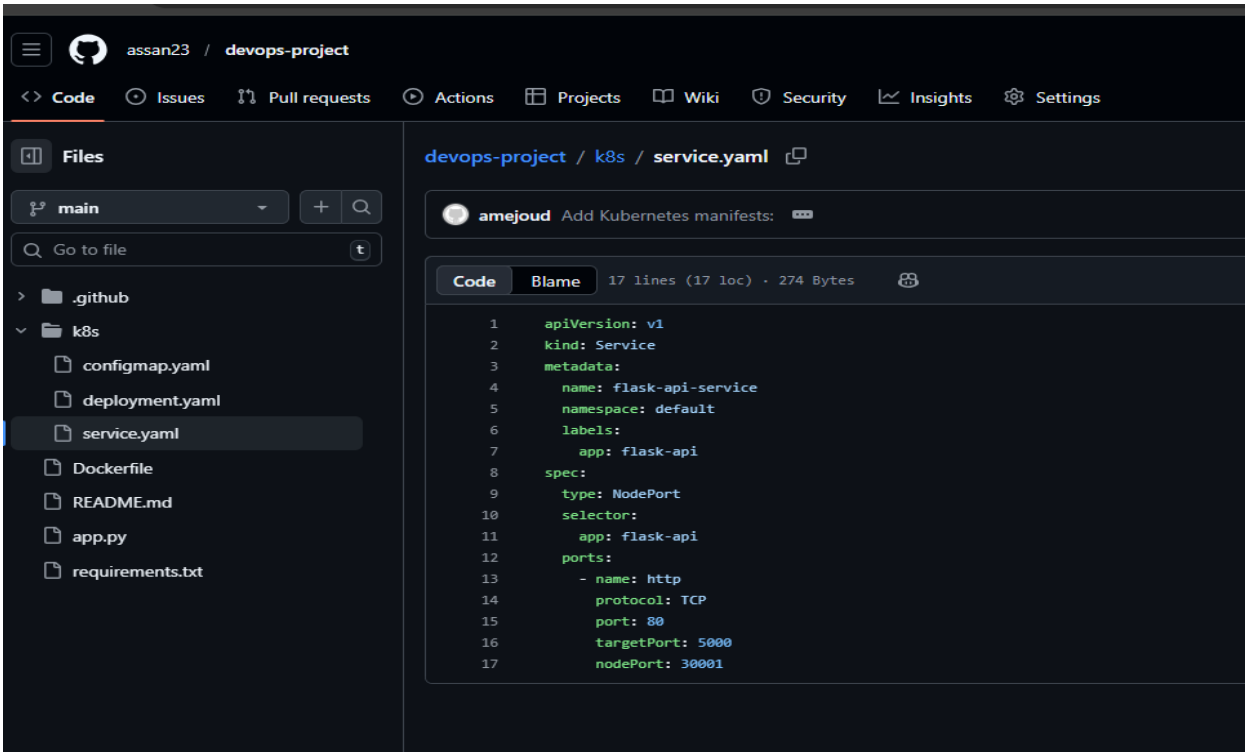


Figure 18Exposition du service Kubernetes test-app via Minikube

ConfigMap

Le `ConfigMap` permet de stocker des données de configuration séparées du code applicatif, ce qui favorise une meilleure gestion des environnements.

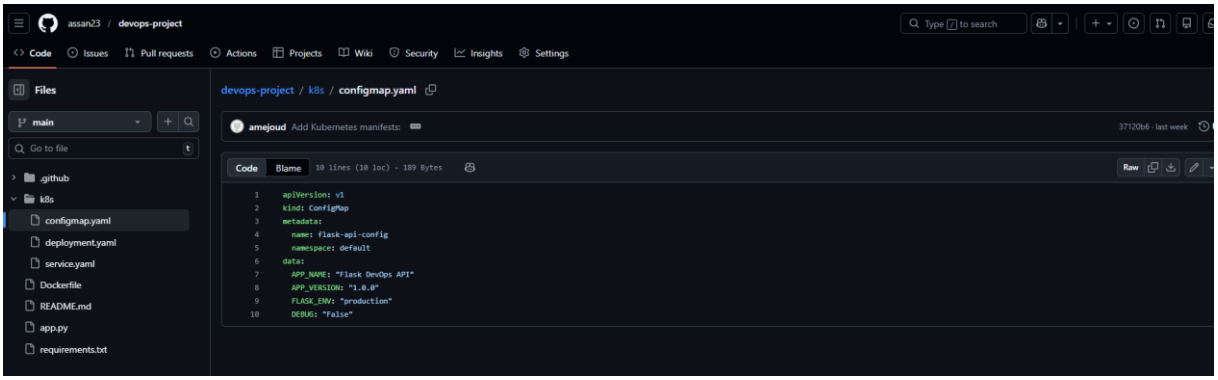


Figure 19 Manifest Kubernetes ConfigMap pour la configuration de l'application

6.3 Déploiement sur le cluster

Les commandes de vérification mentionnées incluent :

- `kubectl get pods`
  `kubectl get svc`

Ces commandes permettent de vérifier respectivement l'état des pods et des services.

6.4 Image Docker utilisée

L'image Docker de l'application est publiée sur Docker Hub sous le compte :

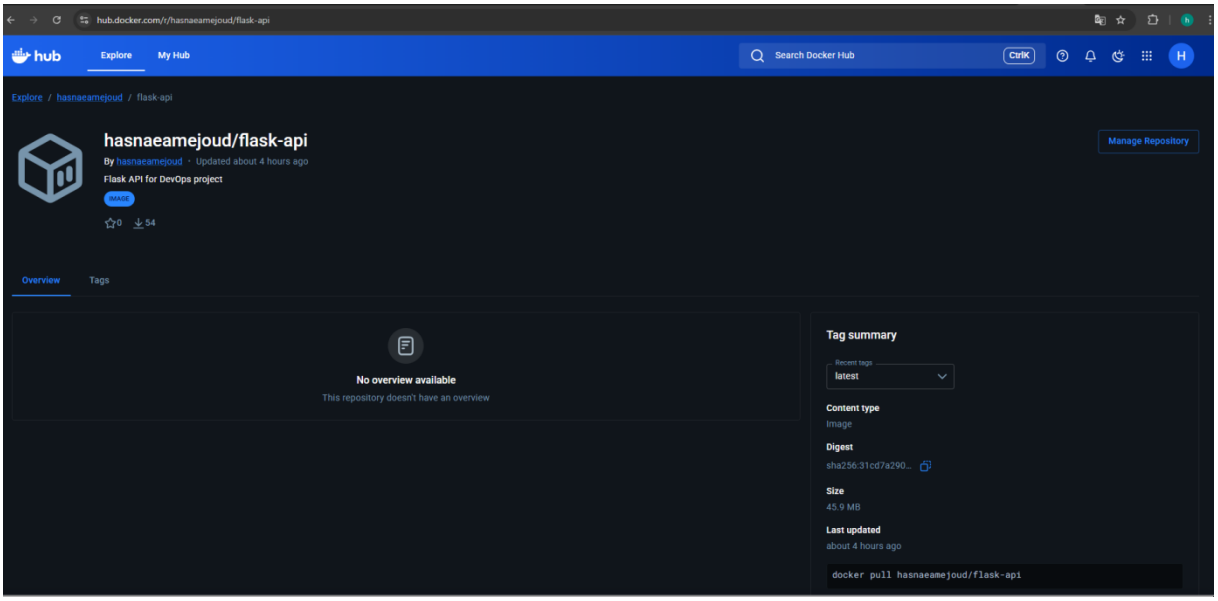


Figure 20 Publication de l'image Docker hasnaeamejoud/flask-api

hasnaeamejoud/flask-api

Elle est publique, ce qui permet au cluster de la télécharger sans authentification.

6.5 Problème rencontré

Une erreur de type **ImagePullBackOff** a été observée lors d'un déploiement. Cette erreur signifie que Kubernetes ne parvient pas à télécharger l'image, à cause d'une image inexistante, d'un mauvais nom ou d'un problème d'accès.

Cela montre l'importance de vérifier :

- le nom de l'image
- la disponibilité sur Docker Hub
- les droits d'accès si l'image est privée

6.6 Validation du service

Le service de type NodePort exposait un port externe au niveau du cluster. Il était également nécessaire de vérifier que le Security Group AWS autorise le trafic entrant sur ce port.

6.7 Tests applicatifs sur Minikube

```
PS C:\Users\ACER\devops-project> minikube service test-app
```

NAMESPACE	NAME	TARGET PORT	URL
default	test-app	8080	http://192.168.49.2:30777

```
🔗  Starting tunnel for service test-app.
```

NAMESPACE	NAME	TARGET PORT	URL
default	test-app		http://127.0.0.1:58866

```
🎉  Opening service default/test-app in default browser...
❗  Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Figure 21Exposition du service Kubernetes test-app via la commande Minikube

```
127.0.0.1:58866

Request served by test-app-684666577f-c4d8l

HTTP/1.1 GET /

Host: 127.0.0.1:58866
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
Connection: keep-alive
Sec-Ch-Ua: "Not:A-Brand";v="99", "Google Chrome";v="145", "Chromium";v="145"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
```

Figure 22 Vérification du fonctionnement de l'application déployée après exposition du service

```
test-app    NodePort    10.105.240.142    <none>        8080:30777/TCP    25h
PS C:\Users\ACER> minikube service hello-minikube
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-minikube	8080	http://192.168.49.2:30382

```
🔗  Starting tunnel for service hello-minikube.
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-minikube		http://127.0.0.1:59209

```
🎉  Opening service default/hello-minikube in default browser...
❗  Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Figure 23 Accès au service hello-minikube exposé sur le cluster

*

```
127.0.0.1:59209

Hostname: hello-minikube-6fcb694474-szvqr

Pod Information:
        -no pod information available-

Server values:
        server_version=nginx: 1.13.3 - lua: 10008

Request Information:
        client_address=10.244.0.1
        method=GET
        real_path=/
        query=
        request_version=1.1
        request_scheme=http
        request_uri=http://127.0.0.1:8080/

Request Headers:
        accept=text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
        accept-encoding=gzip, deflate, br, zstd
        accept-language=fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
        connection=keep-alive
        host=127.0.0.1:59209
        sec-ch-ua=&quot;Not:A-Brand&quot;;v=&quot;99&quot;, &quot;Google Chrome&quot;;v=&quot;145&quot;, &quot;Chromium&quot;;v=&quot;145&quot;
        sec-ch-ua-mobile=?0
        sec-ch-ua-platform=&quot;Windows&quot;
        sec-fetch-dest=document
        sec-fetch-mode=navigate
        sec-fetch-site=none
        sec-fetch-user=?1
        upgrade-insecure-requests=1
        user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

Request Body:
        -no body in request-
```

Figure 24 Validation du déploiement Kubernetes et informations du Pod via navigateur

Le document de déploiement mentionne deux validations :

- `minikube service test-app`
- `minikube service hello-minikube`

Les réponses obtenues montrent que :

- les pods sont en état de fonctionnement
- les services sont correctement configurés
- les applications répondent aux requêtes HTTP

6.8 Conclusion

Kubernetes a permis de structurer le déploiement de l'application de manière claire et reproductible. Grâce aux manifests, il devient possible de redéployer l'application rapidement sur un cluster et de contrôler son exposition réseau.

● Chapitre 7: Mise en place du pipeline CI/CD

7.1 Introduction

Dans une démarche DevOps, la chaîne CI/CD permet d'automatiser les étapes de build, test et livraison de l'application. Le sujet du projet impose précisément la mise en place d'un tel pipeline.

7.2 Outil choisi

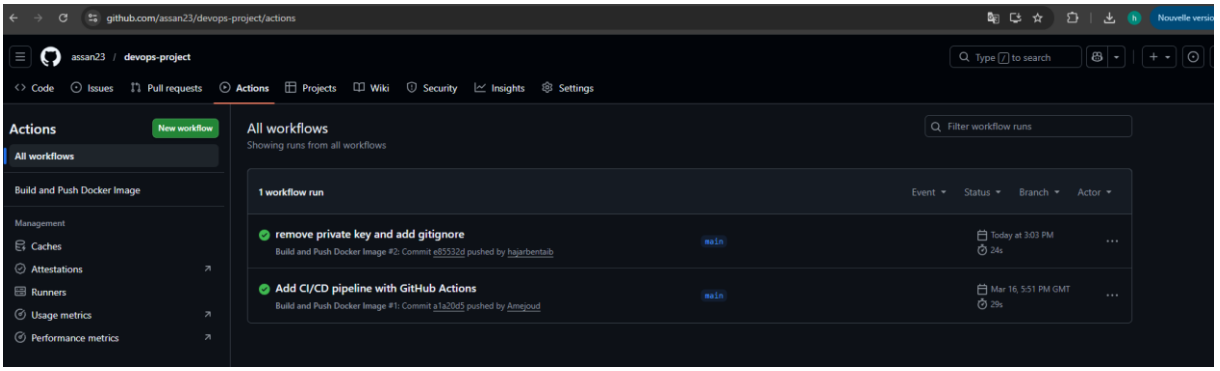


Figure 25 Exécution du pipeline CI/CD avec GitHub Actions pour le build et le déploiement de l'image Docker

Le pipeline a été implémenté avec **GitHub Actions**, déclenché automatiquement à chaque `push` sur la branche `main`.

7.3 Étapes du pipeline

Les étapes mentionnées sont :

- **Checkout** : récupération du code source
- **Login Docker Hub** : authentification auprès du registre
- **Build** : construction de l'image Docker
- **Push** : publication de l'image sur Docker Hub

7.4 Structure du dépôt

Le dépôt GitHub contient notamment :

- `.github/workflows/deploy.yml`
- `Dockerfile`
- `app.py`
- `requirements.txt`
- `k8s/deployment.yaml`
- `k8s/service.yaml`
- `k8s/configmap.yaml`

Cette organisation sépare clairement :

- le code source
- la conteneurisation
- l'infrastructure Kubernetes
- l'automatisation CI/CD

7.5 Apport du pipeline

L'automatisation du build et du push permet :

- de gagner du temps
- de réduire les erreurs manuelles
- de garantir la disponibilité d'une image Docker à jour
- de préparer le déploiement futur sur Kubernetes

7.6 Conclusion

Le pipeline GitHub Actions constitue une étape essentielle du projet, car il met en œuvre les principes d'intégration continue et prépare la voie à un déploiement continu automatisé.

●Chapitre 8: Validation et tests

8.1 Validation de l'application locale

Les tests locaux de l'API Flask ont montré que :

- l'endpoint principal renvoie un message de bienvenue
- l'endpoint `/health` renvoie un statut `OK`

8.2 Validation du conteneur Docker

Le conteneur a été exécuté avec succès via Docker, ce qui confirme :

- la validité du Dockerfile
- l'installation correcte des dépendances
- le bon lancement du serveur Flask

8.3 Validation Kubernetes

Les tests effectués via `minikube service` ont confirmé que les services fonctionnent et que les pods répondent aux requêtes HTTP.

8.4 Validation du pipeline CI/CD

L'historique des workflows montre que le pipeline GitHub Actions s'exécute suite aux mises à jour du dépôt et publie l'image Docker sur Docker Hub.

8.5 Interprétation des résultats

Les validations montrent que la chaîne globale fonctionne correctement :

- code applicatif opérationnel
- image Docker construite
- image publiée dans un registre
- déploiement Kubernetes défini
- pipeline CI/CD fonctionnel

Conclusion générale et perspectives

Ce projet a permis de mettre en œuvre une infrastructure DevOps complète conforme au sujet proposé. Le cahier des charges imposait la création d'une infrastructure cloud, l'utilisation de Terraform et Ansible, la conteneurisation d'une application, son déploiement sur Kubernetes et l'automatisation de la chaîne CI/CD.

Dans ce cadre, une application REST simple basée sur Flask a été développée et conteneurisée avec Docker. Une infrastructure AWS composée de deux machines virtuelles, **kubernetes-master** et **kubernetes-worker**, a été provisionnée via Terraform. Les manifests Kubernetes ont permis de structurer le déploiement de l'application, tandis que GitHub Actions a automatisé la construction et la publication de l'image Docker sur Docker Hub.

Au-delà de l'aspect technique, ce projet a permis de comprendre l'intérêt d'une approche DevOps fondée sur l'automatisation, la reproductibilité et l'intégration de plusieurs outils complémentaires. Il constitue ainsi une mise en pratique concrète des concepts étudiés dans le cadre du Master DSBD & IA.

References

- · HashiCorp. Terraform Documentation.
  Disponible sur : https://developer.hashicorp.com/terraform/docs
- · Amazon Web Services. Amazon EC2 Documentation.
  Disponible sur : https://docs.aws.amazon.com/ec2/
- · Docker Inc. Docker Documentation.
  Disponible sur : https://docs.docker.com/
- · Kubernetes Authors. Kubernetes Documentation.
  Disponible sur : https://kubernetes.io/docs/home/
- · Flask Documentation. Flask Web Framework.
  Disponible sur : https://flask.palletsprojects.com/
- · GitHub. GitHub Actions Documentation.
  Disponible sur : https://docs.github.com/en/actions
- · Red Hat. Ansible Documentation.
  Disponible sur : https://docs.ansible.com/
- · Turnbull, J. (2014). The Docker Book.
- · Burns, B., Beda, J., Hightower, K. (2019). Kubernetes: Up and Running. O'Reilly Media.
- · Humble, J., Farley, D. (2010). Continuous Delivery. Addison-Wesley